

Alight International Multi-Vendor Marketplace

Overview

Alight International Multi-Vendor Marketplace is a high-performance, production-oriented e-commerce platform designed for multi-vendor retail operations.

Architecture

The application is designed as a **Modular Monolith**:

- **Backend:** Java 21 LTS, Spring Boot 3.3.x, Spring Security, Spring Data JPA, Flyway migrations, PostgreSQL.
- **Frontend:** Next.js (App Router), React, TypeScript, Tailwind CSS, Centralized API Client.
- **API Standard:** RESTful `/api/v1/` endpoints with unified JSON responses and error structures.

Project Structure

```
TEXT

alight.com/
├── backend/                                # Spring Boot 3.x Modular Monolith
│   ├── pom.xml
│   └── src/
│       ├── main/java/com/alight/marketplace/
│       │   ├── AlightMarketplaceApplication.java
│       │   ├── common/                    # Responses, errors, validations, constants
│       │   ├── config/                   # Security, CORS, OpenAPI documentation
│       │   └── modules/                  # Domain modules (auth, vendor, product, etc.)
│       └── main/resources/
│           ├── application.yml
│           └── db/migration/             # Flyway SQL migrations
├── frontend/                             # Next.js TypeScript Frontend
│   ├── src/
│   │   ├── app/
│   │   │   ├── (storefront)/            # Customer shopping shell
│   │   │   ├── (vendor)/                # Vendor portal shell
│   │   │   └── (admin)/                 # Admin operational shell
│   │   ├── components/                  # Reusable UI & Layout components
│   │   ├── services/                   # Centralized typed API clients
│   │   └── styles/                      # Brand design tokens & Tailwind config
├── docker/                              # Database & container configs
├── docker-compose.yml
├── .env.example
└── README.md
```

Prerequisites

- Java 21 LTS or newer

- **Maven 3.9+**
- **Node.js 20+ & npm 10+**
- **PostgreSQL 15+ or Docker**

Getting Started

1. Environment Configuration

Copy `.env.example` to `.env` in the root:

BASH

```
cp .env.example .env
```

2. Start Database

Using Docker Compose:

BASH

```
docker compose up -d postgres
```

Or use a local PostgreSQL instance listening on port `5432` with database `alight_db`.

3. Start Backend

BASH

```
cd backend  
mvn clean spring-boot:run
```

- Health Check: `GET http://localhost:8080/api/v1/health`
- OpenAPI Swagger UI: `http://localhost:8080/swagger-ui.html`
- API Documentation: `http://localhost:8080/v3/api-docs`

4. Start Frontend

BASH

```
cd frontend  
npm install  
npm run dev
```

- Customer Storefront: `http://localhost:3000`
- Vendor Portal: `http://localhost:3000/vendor`
- Admin Portal: `http://localhost:3000/admin`

Development Stages (20-Stage Architecture)

- **Stage 1: Project Foundation** (Production-Grade: Live DB diagnostics, MDC correlation IDs, security

headers, resilient API client with timeout & auto-token fallback, branded 404/500/loading error boundaries)

- **Stage 2: Database / ERD** (71 tables, Flyway migrations V1-V30, PostgreSQL 18, automated updated_at triggers, pg_trgm search GIN indexes, @Version optimistic locking)
- **Stage 3: RBAC & Permission Matrix** (4-Tier Roles: Customer, Vendor, Admin, Super Admin; 45 granular permissions, JWT embedded permissions, ResourceSecurityService multi-tenant isolation, Admin RBAC management APIs)
- **Stage 4: API Specification & Documentation** (Production OpenAPI 3.0, Multi-Environment Servers, BearerAuth JWT, 6 Grouped Swagger UI Modules, STAGE_4_API_SPECIFICATION.md manual on [/swagger-ui.html](#))
- **Stage 5: Backend Modular Monolith** (Spring Boot 3.3.4, Java 21 LTS, 28 Domain Modules, In-Memory Domain Event Bus, ThreadPoolTaskExecutor with MDC Correlation, PagedResponse & 409 Conflict Handling)
- **Stage 6: Authentication & Session Management** (JWT, refresh tokens, BCrypt hashing)
- **Stage 7: Vendor Management** (Registration, KYC verification, Admin approvals, status transitions)
- **Stage 8: Product Catalog** (Modular kitchen, wardrobe, bathroom hardware; hierarchical categories, variants)
- **Stage 9: Inventory** (Warehouse stock tracking, reservation engine, oversell prevention)
- **Stage 10: Shopping Cart** (Multi-vendor cart items, currency conversions)
- **Stage 11: Multi-Vendor Orders** (Parent order #10001 → Vendor fulfillment orders #10001-A, #10001-B)
- **Stage 12: Payments** (Razorpay webhook verification, signature security, COD threshold validation)
- **Stage 13: Logistics & Shipping** (Multi-carrier abstraction, zone/weight matrix, packing slips, tracking)
- **Stage 14: Returns, Refunds & RTO** (7-day window, photo verification, reverse logistics, RTO separation)
- **Stage 15: Settlements & Commission** (Configurable commissions, ledger calculation, hold periods, payouts)
- **Stage 16: Admin Dashboard & Moderation** (Vendor approvals, catalog control, disputes, settlement ledger)
- **Stage 17: Vendor Dashboard & Store Portal** (Store profile, product management, fulfillment, earnings)
- **Stage 18: Customer Marketplace Experience** (Modern storefront, instant search with facets, multi-step checkout)
- **Stage 19: Security Hardening & Audit Logging** (Immutable audit_logs table, transaction-isolated recording)
- **Stage 20: Containerized Production Deployment** (Multi-stage Dockerfiles, docker-compose.prod.yml)*